Caleb Fletcher
Hunter Clan
Gabriel Holman
CSC 2400-003

Project Proposal

1. **Problem**
    a. **What is the problem?**
    The problem is determining which CPU scheduling algorithm is best for executing a set of tasks efficiently.
    b. **What information is given as input?**
    The input includes a set of CPU tasks, execution times, arrival times and priorities along with the scheduling algorithms being tested, those being Brute-Force, Genetic, and Greedy Algorithms.
    c. **What constitutes a valid solution?**
    A valid solution is a schedule that assigns all tasks an execution order according to the rules of scheduling algorithms being used.
    d. **What makes one solution better than another?**
    One solution is better than the other if it completes tasks more efficiently, reduces wait times, lowers turnaround times and uses the CPU resources efficiently.
    e. **Is the goal to minimize or maximize something?**
    The goal is to minimize average waiting time, turn around time and scheduling overhead while maintaining efficient CPU utilization.
    f. **Where does this problem occur in the real world?**
    CPU scheduling occurs in operating systems that manage processes on computers, cell phones, and servers. It helps determine which tasks receive CPU time so that it can run efficiently.

2. **Why This Problem?**
    We chose this problem simply because it was something we found interesting and wanted to explore. Solving this problem efficiently matters because modern computing environments rely heavily on fast, balanced task execution. Investigating algorithmic trade-offs in CPU scheduling allows us to explore how theoretical optimization directly impacts real-world system performance.

3. **Brute-Force Approach - Gabriel**
    a. **What Possibilities Must be Examined?**
    An operating system will examine whatever the current state of its CPU task set is and it will then go through every combination of tasks to be completed under differing policies. If, for instance, there are five tasks and five scheduling policies, then there are 3,125 different ways to complete all tasks, where that number has a high possibility to contain more than one fastest method. This is found by raising the amount of tasks to the power of however many policies the operating system is ready to implement. For each combination, the operating system will calculate how long it would take to complete each task, and once all possibilities have been tested, it will execute the optimal path.
    b. **How many solutions exist as the input becomes larger?**
    Since the growth formula used $p = n^r$ — where p is the amount of possibilities, n is the number of objects in the set, and r is the number of objects being arranged is exponential, the number of solutions explodes as n becomes larger, i.e., $O(c^n)$.
    c. **Why Might Brute Force Become Impractical?**
    While finding the optimal method for scheduling CPU tasks is the main goal, it would be detrimental to implement a brute force algorithm for this problem because the operating system would spend far more time thinking about how to schedule tasks rather than actually scheduling tasks. Also, if there are more than one most optimal ways for completing a task set, the ideal way for the operating system to handle this case would be to stop looking for the best path at this point, and to just execute it. But a brute force algorithm can not know that one way is best without first going through all possibilities.

4. **Genetic Algorithms - Hunter**
    Skipping for now as per instructions.

5. **Greedy Algorithm - Caleb**
    a. **How it Works**
    Greedy algorithms solve the scheduling problem by selecting the next task to execute based on a specific local criterion (such as shortest time, earliest deadline, or highest priority) without looking ahead at future tasks. This is repeated until all tasks have been scheduled and processed.
    b. **Why It Is Appropriate**
    Operating systems require fast, efficient decision-making under constant workload pressure. Rather than spending excessive time computing global optimizations, a greedy strategy provides a fast, deterministic way to construct schedules in real-time.
    c. **How It Differs from Brute-Force**
    Unlike brute-force execution, which explores every single possible permutation of task scheduling to find the absolute optimum, a greedy algorithm makes a single, rapid sequence of decisions based on local heuristics.

6. **Research Question**

   A CPU task scheduler can use many different scheduling policies based on process behavior and system load. For any given set of tasks, how should an operating system decide which scheduling policy is the best to use for the processor's set of tasks? Given that this set of tasks for the processor is dynamic, how can the operating system balance CPU overhead with adapting to workload changes?

7. **Experimental Plan**

   The best way to compare algorithms is to give each algorithm the exact same task set and record how long each algorithm takes to schedule all tasks within their set. In addition to time spent scheduling, the CPU execution time must also be considered. One algorithm may have scheduled tasks the fastest, but did it schedule them in a way that minimizes CPU execution time the most? Another algorithm may have produced the most optimal arrangement of tasks, but spent an eternity in doing so. Therefore, the best choice can not be obtained by examining either of the times. Instead, the best choice would be whichever algorithm has the lowest combined time out of all tested algorithms.

   However, if this test set contained mostly parallel tasks, then the solution would likely apply only to other parallel task sets. Others may contain different types of tasks where the most efficient schedule is different from the parallel set. These differences must be recorded alongside the most common task in that set to serve as the set's identifier. A set consisting of multimedia tasks may be more efficiently scheduled by a brute force algorithm, while a set consisting of real-time tasks may be more efficiently scheduled by a genetic algorithm.

   So, the full answer to the CPU task scheduling problem would look like a list where each element is every type of task that can be executed by a CPU, and that element is correlated to either a genetic, brute force, or greedy algorithm. This list would read as: "If a task set contains mostly T types of tasks, then use algorithm A."

8. **Expected Challenges**

   a. Managing the exponential increase in runtime for brute-force execution on large task inputs. We need to carefully cap the maximum problem size for brute-force trials while ensuring the test set remains large enough to generate meaningful comparison data.

   b. Generating synthetic task sets with realistic duration/arrival distributions that fairly reflect actual computing workloads. Artificial or purely uniform task sets might produce misleading algorithmic performance results that do not reflect real-world operating system workloads or multi-core distribution scenarios.

9. **Sources**

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley. https://www.os-book.com/OS10/

MIT OpenCourseWare. (2015). *Design and Analysis of Algorithms*. Massachusetts Institute of Technology. https://ocw.mit.edu/courses/6-046j-design-and-analysis-of-algorithms-spring-2015/

Eiben, A. E., & Smith, J. E. (2015). *Introduction to Evolutionary Computing* (2nd ed.). Springer. https://doi.org/10.1007/978-3-662-44874-8

10. **Group Expectations**
    a. **Work Distribution**
       Each member will focus on one algorithm.
    b. **Meeting Times**
       We will meet in person at least twice a week during class time to discuss our progress. Most other work will be conducted virtually.
    c. **Communication**
       Microsoft Teams will be the main form of communication.
    d. **Conflict Resolution**
       Efforts will be made to contact one another on Microsoft Teams first, and in person during class if need be. Instructor involvement will be a last resort.